

# DIGITAL SYSTEMS

## LECTURE 13 DEVICE DRIVERS

Mark van der Wilk  
Hilary Term 2026



Department of  
COMPUTER  
SCIENCE

Following the course of past years by Mike Spivey

## Lab 4

Lab 4 spec is now updated to include specific instructions for S/S+.

As usual, not finishing everything required for S will not preclude you from getting an S, if you show you engaged properly.

Doing the labs is a great preparation for exams though.

# List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
- seeming as each program is running simultaneously,
- **and doesn't require the programmer to know when to disable interrupts in order to guarantee correct code.**

## ? How can we safely respond to outside events?

- How can we write code to respond to interrupts (drivers)?
  - How can events be communicated to programs?
- 
- Take advantage of the safety of messages, by making interrupts send messages from hardware.
  - Interrupt from peripheral will send messages to *device driver*.
  - Other processes communicate with hardware through device driver, by sending messages.
  - Device driver is a process that serves messages from hardware/software in a loop.

# Serial Output

```
void serial_putc(char ch) {  
    message m;  
    m.int1 = ch;  
    send(SERIAL, PUTC, &m);  
}
```

- Now interacts with hardware via the driver, rather than directly.
- Request message sent to SERIAL driver.
- Process yields to the OS, and waits if driver is not yet ready.

# Driver Process

```
void serial_task(int arg) {  
    static char txbuf[NBUF]; // Note: State is local to driver!  
    int bufin, bufout, bufcount; message m; char ch;  
    serial_setup();  
    while (1) { // Loop serves request  
        receive(ANY, &m);  
        switch (m.m_type) {  
            ...  
        }  
    }  
}
```

# Setting Things Up

```
void serial_setup(void) {  
    ...  
    connect(UART_IRQ);  
    enable_irq(UART_IRQ);  
    UART_INTENSET = BIT(UART_INT_TXDRDY);  
}
```

# Handling PUTC Messages

```
while (1) {  
    receive(ANY, &m);  
    switch (m.m_type) {  
        case PUTC:  
            ch = m.int1;  
            txbuf[bufin] = ch; ... // rest of circular buffer  
            break;  
        ...  
    }  
}
```

- Buffer variables are local, so no other process can interfere.
- No need for volatile or to disable interrupts.



# Questions

Several questions remain to be answered:

1. How do characters get out of the buffer and into the UART?
2. What happens about interrupts?
3. What happens when someone wants to send a PUTC message but the buffer is full?

# Handling Interrupts

Remember: Hardware sends INTERRUPT message.

Inside loop, inside switch...

```
case INTERRUPT:
    if (UART_TXDRDY) {
        txidle = 1;
        UART_TXDRDY = 0;
    }
    clear_pending(UART_IRQ);
    enable_irq(UART_IRQ);
    break;
```

# Responding to Events

Send character, if possible, after any message.

```
while (1) {  
    receive(ANY, &m);  
    switch (m.m_type) {  
        ...  
    }  
    if (txidle && bufcount > 0) {  
        UART_TXD = txbuf[bufout]; ...  
        txidle = 0;  
    }  
}
```



**Use message mechanism to synchronise processes!**

# Full Driver

```
while (1) {  
    if (bufcount == NBUF)  
        receive(INTERRUPT, &m);  
    else  
        receive(ANY, &m);  
    switch (m.type) {  
    case INTERRUPT:  
        if (UART_TXDRDY) {  
            txidle=1;UART_TXDRDY=0;}  
        clear_pending(UART_IRQ);  
        enable_irq(UART_IRQ);  
        break;
```

```
    case PUTC:  
        txbuf[bufin] = m.int1;  
        bufin = wrap(bufin+1);  
        bufcount++;  
        break;  
    }  
    if (txidle && bufcount>0) {  
        UART_TXD = txbuf[bufout];  
        bufout = wrap(bufout+1);  
        bufcount--;  
        txidle = 0;  
    } } }
```

# Omitted Here

Lab 4 has a more elaborate serial driver

- Supports both output and input with echoing and line editing.
- All UART initialisation details are filled in.
- There's an alternative interface `print_buf` that overcomes the one-message-per-character bottleneck.

# Bottleneck

How many context switches per character?

- $\sim 20\mu\text{s}$  per context switch
- UART 9600 baud ( $\sim 100\mu\text{s}$  per bit)  $\Rightarrow$  1k characters/s
- Can use up to  $10^5\mu\text{s}$  of CPU time per second! 10% of CPU time!
- Allow `printf()` to accumulate characters in a buffer, and send them all at once.
- But... shared memory!

# Standard Interrupt Handler

```
void default_handler(void) {  
    int irq = active_irq();  
    int task = os_handler[irq];  
    disable_irq(irq);  
    interrupt(task);    // Send an interrupt message to process  
}
```

- Driver can be ready to accept message  
→ context switch
- Driver can not be ready to accept message  
→ deliver interrupt message later, at next available opportunity

Why do we need `disable_irq(...)`?



# Example

Heart & Primes example. Can we find sequence of events? When we switch contexts, when CPU sleeps/wakes?

- Look at `ex-heart.c`.
- Look at `serial.c`

If time, look at `print_buf()`.

# Summary

- Take advantage of the safety of messages, by making interrupts send messages from hardware.
- Interrupt from peripheral will send messages to *device driver*.
- Device driver is a process that serves messages from hardware/software in a loop.

## ? How does the OS switch tasks?

- How does it decide what to run next?
- How does it move messages?
- How does it actually do the context switch?